ITC-C508 Lab Exercise WW1 and WW2 (Optimization and GPU Testing)

Submitted by Group: Cache

Members:

- Sasil, Perlin Precious (Group Leader)
- Laylo, Hannah Alexa
- Tuguinay, Nanelle

Submitted to: Doc. Rodolfo Raga

Introduction

- The objective of this experiment was to determine the optimal Retrieval-Augmented Generation (RAG) parameters for the Barangay VAW Desk Handbook and evaluate the difference in execution time between CPU and GPU processing. The experiment used the Barangay VAW Desk Handbook, as the primary source of information for the chatbot. Five configurations (A–E) were tested by varying the chunk size, chunk overlap, and retrieval top-k values to determine which combination provided the most suitable retrieval performance and generated answers for the dataset.
- In all the configurations tested during the experiment, the source document, test questions, language model, and generation parameters remained the same to provide a reliable point of comparison. The indexing time, retrieval time, response generation time, and total time taken to execute were also considered where applicable. In addition, the answers produced were examined in relation to their accuracy, relevancy to the retrieved documents, and compatibility with the source document. Both CPU and GPU executions were undertaken to find out the comparative processing times of each and the benefits that can be derived from the use of GPU. The findings from these experiments were used to select the best RAG configuration for the Barangay VAW Desk Handbook chatbot.

DATASET: Barangay VAW Desk Handbook 2023

- The first dataset used in this experiment is the *Barangay VAW Desk Handbook.* The handbook provides information and guidelines for Barangay Violence Against Women (VAW) Desk officers in responding to cases of violence against women and their children. It covers the roles and responsibilities of VAW Desk officers, initial assistance and assessment, proper documentation, interview procedures, referral mechanisms, and information about the rights and legal remedies available to victims, including protection orders.
- For this experiment, the handbook was processed and converted into text that could be divided into smaller chunks for embedding and retrieval. These chunks were indexed in a vector database to allow the chatbot to retrieve relevant passages based on users' questions. The retrieved passages serve as contextual references for generating answers, helping ensure that responses are grounded in the information provided by the handbook rather than relying solely on the language model's pretrained knowledge.
- This dataset was used to evaluate how different chunk sizes, overlap values, and retrieval top-k settings affect retrieval relevance, answer correctness, response generation time, and overall chatbot performance. The experiment also compared CPU and GPU execution to examine the effect of hardware acceleration on processing time.

Executable RAG Implementation

- This section presents the complete and executable RAG implementation for the Barangay VAW Desk Handbook chatbot. It includes document processing, text chunking, embedding generation, vector database creation, information retrieval, and answer generation. The implementation also supports testing different RAG parameter configurations and benchmarking CPU and GPU execution to evaluate retrieval performance and processing time.

```
# ============================================================
# RAG DOCUMENT PROCESSING PIPELINE
```

```
# PDF EXTRACTION, CLEANING, CHUNKING, EXCEL EXPORT,
# EMBEDDINGS, FAISS, AND RETRIEVAL
# Google Colab implementation
# ============================================================


# ============================================================
# 1. INSTALL REQUIRED LIBRARIES
# ============================================================

!pip install -q faiss-cpu sentence-transformers pypdf transformers


# ============================================================
# 2. IMPORT LIBRARIES
# ============================================================

from google.colab import files
from pypdf import PdfReader
from sentence_transformers import SentenceTransformer

import faiss
import numpy as np
import pandas as pd
import pickle
import re
import os

from openpyxl.styles import Font, PatternFill, Alignment


# ============================================================
# 3. INDEXING PARAMETERS
# ============================================================

CHUNK_SIZE = 800
CHUNK_OVERLAP = 80
TOP_K = 3

EMBEDDING_MODEL_NAME = "all-MiniLM-L6-v2"

# Ignore pages with almost no usable extracted text.
# This helps prevent page numbers and isolated symbols
# from becoming standalone chunks.
MIN_PAGE_TEXT_LENGTH = 20

# Avoid creating tiny final chunks when possible.
MIN_TAIL_LENGTH = 50


# ============================================================
# 4. VALIDATE PARAMETERS
# ============================================================
```

```
if CHUNK_SIZE <= 0:
    raise ValueError("CHUNK_SIZE must be greater than 0.")

if CHUNK_OVERLAP < 0 or CHUNK_OVERLAP >= CHUNK_SIZE:
    raise ValueError(
        "CHUNK_OVERLAP must be >= 0 and smaller than CHUNK_SIZE."
    )

if TOP_K <= 0:
    raise ValueError("TOP_K must be greater than 0.")

print("=" * 70)
print("RAG INDEXING CONFIGURATION")
print("=" * 70)
print("Chunk size             :", CHUNK_SIZE)
print("Chunk overlap          :", CHUNK_OVERLAP)
print("Top-K                  :", TOP_K)
print("Embedding model        :", EMBEDDING_MODEL_NAME)
print("Minimum page text      :", MIN_PAGE_TEXT_LENGTH)
print("Minimum tail length    :", MIN_TAIL_LENGTH)


# ============================================================
# 5. TEXT CLEANING FUNCTIONS
# ============================================================

def clean_pdf_text(text):
    """
    Remove illegal XML control characters that may appear
    in PDF-extracted text.

    Preserve normal whitespace and readable Unicode text.
    """

    if not isinstance(text, str):
        return ""

    # Remove control characters that Excel/XML cannot store.
    text = re.sub(r"[\x00-\x08\x0B\x0C\x0E-\x1F]", "", text)

    # Normalize whitespace into single spaces.
    text = re.sub(r"\s+", " ", text).strip()

    return text


def clean_excel_text(text):
    """
    Extra safety check before writing text to Excel.
    """

    if not isinstance(text, str):
        return text
```

```
    return re.sub(r"[\x00-\x08\x0B\x0C\x0E-\x1F]", "", text)


# ============================================================
# 6. UPLOAD PDF DOCUMENT
# ============================================================

print("\n" + "=" * 70)
print("UPLOAD DOCUMENT")
print("=" * 70)

uploaded = files.upload()

if not uploaded:
    raise RuntimeError("No file was uploaded.")

pdf_file = next(iter(uploaded.keys()))

if not pdf_file.lower().endswith(".pdf"):
    raise ValueError("Please upload a PDF file.")

print("\nSelected file:", pdf_file)


# ============================================================
# 7. EXTRACT AND CLEAN PDF TEXT
# ============================================================

print("\n" + "=" * 70)
print("EXTRACTING AND CLEANING PDF TEXT")
print("=" * 70)

reader = PdfReader(pdf_file)

pages = []
empty_pages = []
short_pages = []

for page_number, page in enumerate(reader.pages, start=1):

    try:
        raw_text = page.extract_text() or ""
    except Exception as error:
        print(f"Warning: Could not extract page {page_number}: {err
        raw_text = ""

    page_text = clean_pdf_text(raw_text)

    if not page_text:
        empty_pages.append(page_number)
        continue

    if len(page_text) < MIN_PAGE_TEXT_LENGTH:
        short_pages.append({
```

```
            "page": page_number,
            "text": page_text
        })
        continue

    pages.append({
        "page": page_number,
        "text": page_text
    })


print("Total PDF pages             :", len(reader.pages))
print("Pages with usable text      :", len(pages))
print("Empty/unextractable pages   :", len(empty_pages))
print("Very short pages skipped    :", len(short_pages))

if short_pages:
    print("\nShort pages skipped:")
    for item in short_pages[:15]:
        print(f"Page {item['page']}: {repr(item['text'])}")

if not pages:
    raise RuntimeError(
        "No usable text was extracted. "
        "The PDF may require OCR."
    )


# ============================================================
# 8. CREATE CHUNKS
# ============================================================

def create_chunks(
    pages,
    chunk_size,
    chunk_overlap,
    min_tail_length=50
):
    """
    Split each page independently into overlapping chunks.

    Each chunk retains its original PDF page number.

    If the final remainder is very short, append it to the
    preceding chunk instead of creating a tiny fragment.
    """

    chunks = []
    chunk_id = 0

    step_size = chunk_size - chunk_overlap

    for page_data in pages:
```

```
        page_number = page_data["page"]
        page_text = page_data["text"]

        start = 0
        page_chunk_positions = []

        while start < len(page_text):

            remaining = len(page_text) - start

            # Avoid an extremely short final chunk.
            # Extend the previous chunk to include the remainder.
            if remaining < min_tail_length and page_chunk_positions
                previous_position = page_chunk_positions[-1]
                previous_chunk = chunks[previous_position]

                previous_start = previous_chunk["_start"]
                previous_chunk["text"] = page_text[
                    previous_start:
                ].strip()

                break

            end = min(start + chunk_size, len(page_text))
            chunk_text = page_text[start:end].strip()

            if chunk_text:
                chunks.append({
                    "chunk_id": chunk_id,
                    "page": page_number,
                    "text": chunk_text,
                    "_start": start
                })

                page_chunk_positions.append(len(chunks) - 1)
                chunk_id += 1

            start += step_size

    # Remove internal processing metadata.
    for chunk in chunks:
        chunk.pop("_start", None)

    return chunks


chunks = create_chunks(
    pages,
    CHUNK_SIZE,
    CHUNK_OVERLAP,
    MIN_TAIL_LENGTH
)

if not chunks:
```

```
    raise RuntimeError("No chunks were generated.")


# ============================================================
# 9. DISPLAY CHUNKING RESULTS
# ============================================================

print("\n" + "=" * 70)
print("CHUNKING RESULTS")
print("=" * 70)

chunk_lengths = [len(chunk["text"]) for chunk in chunks]

print("Number of chunks             :", len(chunks))
print("Average characters per chunk :", round(np.mean(chunk_lengths
print("Shortest chunk               :", min(chunk_lengths))
print("Longest chunk                :", max(chunk_lengths))

tiny_chunks = [
    chunk for chunk in chunks
    if len(chunk["text"].strip()) < MIN_TAIL_LENGTH
]

print("Chunks shorter than 50 chars:", len(tiny_chunks))

if tiny_chunks:
    print("\nShort chunks for inspection:")
    for chunk in tiny_chunks[:10]:
        print(
            f"Chunk {chunk['chunk_id']} | "
            f"Page {chunk['page']} | "
            f"{repr(chunk['text'])}"
        )


# ============================================================
# 10. EXPORT CHUNKS TO EXCEL
# ============================================================

print("\n" + "=" * 70)
print("EXPORTING CHUNKS TO EXCEL")
print("=" * 70)

chunk_records = []

for chunk in chunks:
    chunk_records.append({
        "Chunk_ID": chunk["chunk_id"],
        "Source_Document": pdf_file,
        "Page_Number": chunk["page"],
        "Character_Count": len(chunk["text"]),
        "Chunk_Size_Setting": CHUNK_SIZE,
        "Chunk_Overlap_Setting": CHUNK_OVERLAP,
        "Chunk_Text": clean_excel_text(chunk["text"])
    })
```

```
        })

chunks_df = pd.DataFrame(chunk_records)

base_name = os.path.splitext(os.path.basename(pdf_file))[0]
EXCEL_FILE = f"{base_name}_chunks_inspection.xlsx"

with pd.ExcelWriter(EXCEL_FILE, engine="openpyxl") as writer:

    chunks_df.to_excel(
        writer,
        sheet_name="Generated_Chunks",
        index=False
    )

    worksheet = writer.sheets["Generated_Chunks"]

    # Header formatting
    for cell in worksheet[1]:
        cell.font = Font(bold=True, color="FFFFFF")
        cell.fill = PatternFill(
            fill_type="solid",
            fgColor="1F4E78"
        )
        cell.alignment = Alignment(
            horizontal="center",
            vertical="center",
            wrap_text=True
        )

    # Column widths
    column_widths = {
        "A": 12,
        "B": 40,
        "C": 14,
        "D": 18,
        "E": 20,
        "F": 23,
        "G": 80
    }

    for column, width in column_widths.items():
        worksheet.column_dimensions[column].width = width

    # Cell formatting
    for row in worksheet.iter_rows(
        min_row=2,
        max_row=worksheet.max_row
    ):
        for cell in row:
            cell.alignment = Alignment(
                vertical="top",
                wrap_text=(cell.column == 7)
            )
```

```
    worksheet.freeze_panes = "A2"
    worksheet.auto_filter.ref = worksheet.dimensions
    worksheet.row_dimensions[1].height = 32

print("Excel file created:", EXCEL_FILE)
print("Chunks exported   :", len(chunks_df))

print("\nExcel preview:")
print(
    chunks_df[
        ["Chunk_ID", "Page_Number", "Character_Count"]
    ].head(10).to_string(index=False)
)

files.download(EXCEL_FILE)


# ============================================================
# 11. DISPLAY SAMPLE CHUNKS
# ============================================================

print("\n" + "=" * 70)
print("SAMPLE CHUNKS")
print("=" * 70)

for chunk in chunks[:3]:
    print(f"\nChunk ID : {chunk['chunk_id']}")
    print(f"Page     : {chunk['page']}")
    print(f"Characters: {len(chunk['text'])}")
    print("-" * 70)
    print(chunk["text"])


# ============================================================
# 12. LOAD EMBEDDING MODEL
# ============================================================

print("\n" + "=" * 70)
print("LOADING EMBEDDING MODEL")
print("=" * 70)

embedding_model = SentenceTransformer(EMBEDDING_MODEL_NAME)

print("Embedding model loaded:", EMBEDDING_MODEL_NAME)


# ============================================================
# 13. PREPARE CHUNK TEXT
# ============================================================

chunk_texts = [chunk["text"] for chunk in chunks]


#
```

```
# ============================================================
# 14. CREATE DOCUMENT EMBEDDINGS
# ============================================================

print("\n" + "=" * 70)
print("CREATING DOCUMENT EMBEDDINGS")
print("=" * 70)

embeddings = embedding_model.encode(
    chunk_texts,
    convert_to_numpy=True,
    normalize_embeddings=True,
    show_progress_bar=True
).astype("float32")

print("Embedding matrix shape:", embeddings.shape)


# ============================================================
# 15. CREATE FAISS INDEX
# ============================================================

print("\n" + "=" * 70)
print("CREATING FAISS VECTOR STORE")
print("=" * 70)

embedding_dimension = embeddings.shape[1]

index = faiss.IndexFlatIP(embedding_dimension)
index.add(embeddings)

print("Embedding dimension:", embedding_dimension)
print("Vectors stored     :", index.ntotal)


# ============================================================
# 16. CREATE CHUNK LOOKUP
# ============================================================

chunk_lookup = {
    position: chunk
    for position, chunk in enumerate(chunks)
}


# ============================================================
# 17. RETRIEVAL FUNCTION
# ============================================================

def retrieve_documents(query, top_k=TOP_K):

    if not query or not query.strip():
        raise ValueError("The query cannot be empty.")

    if top_k <= 0:
```

```
    if top_k <= 0:
        raise ValueError("top_k must be greater than 0.")

    query_embedding = embedding_model.encode(
        [query],
        convert_to_numpy=True,
        normalize_embeddings=True
    ).astype("float32")

    search_k = min(top_k, index.ntotal)

    scores, indices = index.search(
        query_embedding,
        search_k
    )

    results = []

    for rank, idx in enumerate(indices[0]):

        if idx == -1:
            continue

        chunk = chunk_lookup[int(idx)]

        results.append({
            "rank": rank + 1,
            "chunk_id": chunk["chunk_id"],
            "page": chunk["page"],
            "score": float(scores[0][rank]),
            "text": chunk["text"]
        })

    return results


# ============================================================
# 18. TEST RETRIEVAL
# ============================================================

print("\n" + "=" * 70)
print("TESTING VECTOR STORE")
print("=" * 70)

test_query = input(
    "\nEnter a test question (or press Enter to skip): "
)

if test_query.strip():

    test_results = retrieve_documents(test_query, TOP_K)

    print("\n" + "=" * 70)
    print("RETRIEVED CHUNKS")
    print("=" * 70)
```

```
    print("=" * 70)

    for result in test_results:
        print(f"\nRESULT {result['rank']}")
        print(f"Chunk ID   : {result['chunk_id']}")
        print(f"Page       : {result['page']}")
        print(f"Similarity : {result['score']:.4f}")
        print("-" * 70)
        print(result["text"])


# ============================================================
# 19. SAVE FAISS INDEX
# ============================================================

FAISS_FILE = "rag_faiss.index"

faiss.write_index(index, FAISS_FILE)

print("\nFAISS index saved:", FAISS_FILE)


# ============================================================
# 20. SAVE CHUNKS AND METADATA
# ============================================================

CHUNKS_FILE = "rag_chunks.pkl"

with open(CHUNKS_FILE, "wb") as f:
    pickle.dump(chunks, f)

print("Chunks saved:", CHUNKS_FILE)


# ============================================================
# 21. SAVE CONFIGURATION
# ============================================================

CONFIG_FILE = "rag_config.pkl"

config = {
    "chunk_size": CHUNK_SIZE,
    "chunk_overlap": CHUNK_OVERLAP,
    "top_k": TOP_K,
    "embedding_model": EMBEDDING_MODEL_NAME,
    "source_document": pdf_file,
    "minimum_page_text_length": MIN_PAGE_TEXT_LENGTH,
    "minimum_tail_length": MIN_TAIL_LENGTH
}

with open(CONFIG_FILE, "wb") as f:
    pickle.dump(config, f)

print("Configuration saved:", CONFIG_FILE)
```

```
# ============================================================
# 22. INDEXING COMPLETE
# ============================================================

print("\n" + "=" * 70)
print("INDEXING COMPLETE")
print("=" * 70)

print("Source document :", pdf_file)
print("Number of chunks:", len(chunks))
print("Vectors stored  :", index.ntotal)
print("Chunk size      :", CHUNK_SIZE)
print("Chunk overlap   :", CHUNK_OVERLAP)
print("Embedding model :", EMBEDDING_MODEL_NAME)

print("\nSaved files:")
print(" ", EXCEL_FILE)
print(" ", FAISS_FILE)
print(" ", CHUNKS_FILE)
print(" ", CONFIG_FILE)

print("\nThe cleaned vector store is ready for retrieval.")
```

Show hidden output

```
# ============================================================
# 1. IMPORT LIBRARIES
# ============================================================

from transformers import (
    AutoTokenizer,
    AutoModelForSeq2SeqLM
)

import torch


# ============================================================
# 2. DEVICE AND CONFIGURATION
# ============================================================

device = "cuda" if torch.cuda.is_available() else "cpu"

LLM_NAME = "google/flan-t5-large"

# FLAN-T5 models have a limited input context.
MAX_INPUT_TOKENS = 768
MAX_NEW_TOKENS = 200
RESERVED_TOKEN_MARGIN = 16
```

```
# Retrieve more passages to improve the chance of finding
# information distributed across different handbook sections.
RAG_TOP_K = 3

FALLBACK_ANSWER = (
    "I cannot find the answer in the provided handbook."
)

print("=" * 70)
print("BARANGAY VAW DESK HANDBOOK RAG CHATBOT")
print("=" * 70)

print("Device:", device)
print("Language model:", LLM_NAME)
print("Maximum input tokens:", MAX_INPUT_TOKENS)
print("Maximum output tokens:", MAX_NEW_TOKENS)
print("Retrieval Top-K:", RAG_TOP_K)


# ============================================================
# 3. LOAD THE LANGUAGE MODEL
# ============================================================

print("\nLoading language model...")

tokenizer = AutoTokenizer.from_pretrained(LLM_NAME)

llm = AutoModelForSeq2SeqLM.from_pretrained(LLM_NAME)

llm = llm.to(device)
llm.eval()

print("Language model loaded successfully.")


# ============================================================
# 4. VALIDATE THE EXISTING RETRIEVAL PIPELINE
# ============================================================

required_objects = [
    "retrieve_documents",
    "index",
    "chunks",
    "embedding_model"
]

missing_objects = [
    name for name in required_objects
    if name not in globals()
]

if missing_objects:
    raise RuntimeError(
```

```
        "The indexing pipeline must run first. Missing: "
        + ", ".join(missing_objects)
    )

if index.ntotal == 0:
    raise RuntimeError("The FAISS index contains no vectors.")

print("\nExisting retrieval pipeline detected.")
print("Indexed vectors:", index.ntotal)
print("Available chunks:", len(chunks))


# ============================================================
# 5. TOKEN COUNTING
# ============================================================

def count_tokens(text):
    return len(
        tokenizer.encode(
            text,
            add_special_tokens=True,
            truncation=False
        )
    )


# ============================================================
# 6. PRIORITIZE RETRIEVED DOCUMENTS
# ============================================================

def prioritize_documents(question, retrieved_documents):
    """
    Sort retrieved passages by descending similarity score.

    FAISS uses normalized embeddings and inner product
    in the indexing pipeline.
    """

    return sorted(
        retrieved_documents,
        key=lambda result: result.get(
            "score",
            float("-inf")
        ),
        reverse=True
    )


# ============================================================
# 7. BUILD THE HANDBOOK-SPECIFIC PROMPT
# ============================================================

def create_rag_prompt(question, retrieved_documents):
    """
```

```
        Construct a token-limited prompt for the Barangay
        VAW Desk Handbook.

        Answers must be supported by the retrieved passages.
        """

        ranked_documents = prioritize_documents(
            question,
            retrieved_documents
        )

        instruction = (
            "You are an information assistant about the "
            "Barangay VAW Desk Handbook, 2023 English edition. "
            "Answer the user's question using only the supplied "
            "handbook passages. The question may concern the "
            "role and responsibilities of a Barangay VAW Desk, "
            "handling reports, intake and documentation, "
            "referrals, support services, confidentiality, "
            "coordination, or procedures described in the "
            "handbook. These are possible topics, not facts "
            "that may be assumed without evidence. "
            "Give a complete, clear answer. For multi-part "
            "questions, address each part separately. Use "
            "numbered steps or bullets when appropriate. "
            "Preserve the handbook's terminology and distinguish "
            "required procedures from recommendations when the "
            "passages make that distinction. Do not invent "
            "steps, deadlines, forms, services, responsibilities, "
            "or legal requirements. Do not use outside knowledge. "
            "If the evidence is insufficient, state exactly: "
            "'I cannot find the answer in the provided handbook.' "
            "Mention the PDF page number when available. "
            "Provide information only; do not make decisions "
            "about a specific person's case."
        )

        prompt_prefix = (
            f"Question: {question}\n\n"
            f"Instructions: {instruction}\n\n"
            "Evidence from the handbook:\n"
        )

        prompt_suffix = "\n\nAnswer:"

        prefix_tokens = tokenizer.encode(
            prompt_prefix,
            add_special_tokens=False
        )

        suffix_tokens = tokenizer.encode(
            prompt_suffix,
            add_special_tokens=False
        )
```

```
available_context_tokens = (
    MAX_INPUT_TOKENS
    - len(prefix_tokens)
    - len(suffix_tokens)
    - RESERVED_TOKEN_MARGIN
    - 1
)

if available_context_tokens <= 0:
    raise ValueError(
        "The question and instructions leave insufficient "
        "space for evidence. Shorten the question."
    )

selected_parts = []
remaining_tokens = available_context_tokens

for result in ranked_documents:

    if remaining_tokens <= 0:
        break

    page = result.get("page", "Unknown")
    chunk_id = result.get("chunk_id", "Unknown")
    source_text = result.get("text", "").strip()

    if not source_text:
        continue

    # Preserve source identifiers in the prompt.
    labeled_text = (
        f"[PDF Page {page}; Chunk {chunk_id}]\n"
        f"{source_text}"
    )

    separator_tokens = tokenizer.encode(
        "\n\n",
        add_special_tokens=False
    ) if selected_parts else []

    source_tokens = tokenizer.encode(
        labeled_text,
        add_special_tokens=False
    )

    tokens_available = (
        remaining_tokens - len(separator_tokens)
    )

    if tokens_available <= 0:
        break

    included_tokens = source_tokens[:tokens_available]
```

```
            if not included_tokens:
                continue

            included_text = tokenizer.decode(
                included_tokens,
                skip_special_tokens=True,
                clean_up_tokenization_spaces=False
            )

            selected_parts.append(included_text)

            remaining_tokens -= (
                len(separator_tokens) + len(included_tokens)
            )

            # Stop when a passage has been truncated.
            if len(included_tokens) < len(source_tokens):
                break

        context = "\n\n".join(selected_parts)

        prompt = (
            prompt_prefix
            + context
            + prompt_suffix
        )

        final_token_count = count_tokens(prompt)

        if final_token_count > MAX_INPUT_TOKENS:
            raise ValueError(
                f"Prompt contains {final_token_count} tokens, "
                f"exceeding the configured limit of "
                f"{MAX_INPUT_TOKENS}."
            )

        print("\n--- PROMPT DIAGNOSTICS ---")
        print("Question:", question)
        print("Retrieved passages:", len(retrieved_documents))
        print("Passages included:", len(selected_parts))
        print("Prompt token count:", final_token_count)
        print("Input token limit:", MAX_INPUT_TOKENS)

        print("\n--- EVIDENCE INCLUDED IN PROMPT ---")

        for i, part in enumerate(selected_parts, start=1):
            print(f"\nEvidence {i}:")
            print(part)

        return prompt


    # ============================================================
    # 9. GENERATE THE ANSWER
```

```
# 8. GENERATE THE ANSWER
# ============================================================

def generate_answer(prompt, max_new_tokens=MAX_NEW_TOKENS):
    """
    Generate an answer within the configured output limit.
    """

    inputs = tokenizer(
        prompt,
        return_tensors="pt",
        truncation=False
    )

    input_length = inputs["input_ids"].shape[1]

    print("\n--- GENERATION DIAGNOSTICS ---")
    print("Input tokens:", input_length)
    print("Input limit:", MAX_INPUT_TOKENS)
    print("Maximum new tokens:", max_new_tokens)

    if input_length > MAX_INPUT_TOKENS:
        raise ValueError(
            f"Input contains {input_length} tokens, exceeding "
            f"the configured limit of {MAX_INPUT_TOKENS}."
        )

    inputs = {
        key: value.to(device)
        for key, value in inputs.items()
    }

    with torch.no_grad():
        outputs = llm.generate(
            **inputs,
            max_new_tokens=max_new_tokens,
            do_sample=False,
            num_beams=4,
            early_stopping=True
        )

    answer = tokenizer.decode(
        outputs[0],
        skip_special_tokens=True
    ).strip()

    if not answer:
        return (
            "The model generated an empty answer. "
            "Check the prompt and retrieved passages."
        )

    return answer
```

```
# ============================================================
# 9. COMPLETE RAG QUERY
# ============================================================

def rag_query(question, top_k=RAG_TOP_K):
    """
    1. Retrieve handbook passages.
    2. Build a token-limited prompt.
    3. Generate an answer.
    4. Return answer, sources, and prompt.
    """

    question = question.strip()

    if not question:
        return "Please enter a question.", [], ""

    retrieved_documents = retrieve_documents(
        question,
        top_k=top_k
    )

    if not retrieved_documents:
        return FALLBACK_ANSWER, [], ""

    prompt = create_rag_prompt(
        question,
        retrieved_documents
    )

    answer = generate_answer(prompt)

    if not answer or not answer.strip():
        answer = (
            "The model could not generate an answer. "
            "Please inspect the retrieved passages."
        )

    return answer.strip(), retrieved_documents, prompt


# ============================================================
# 10. CONTINUOUS QUESTION-ANSWERING LOOP
# ============================================================

print("\n" + "=" * 70)
print("CONTINUOUS HANDBOOK QUESTION ANSWERING")
print("=" * 70)

print("Ask questions about the Barangay VAW Desk Handbook.")
print("Type 'exit' to stop.")
print("Type 'sources' to show the last query's retrieved passages."
print("Type 'prompt' to show the last prompt.")
```

```
last_answer = None
last_sources = []
last_prompt = None

while True:

    print()
    user_question = input("Question: ").strip()

    if user_question.lower() == "exit":
        print("\nRAG session terminated.")
        break

    if not user_question:
        print("Please enter a question.")
        continue

    if user_question.lower() == "sources":

        if not last_sources:
            print("No previous query sources are available.")
            continue

        print("\n" + "=" * 70)
        print("SOURCES FROM LAST QUERY")
        print("=" * 70)

        for result in last_sources:

            score = result.get("score", None)
            score_text = (
                f"{score:.4f}"
                if isinstance(score, (int, float))
                else "Unavailable"
            )

            print(
                f"\nRank {result.get('rank', '?')} | "
                f"PDF Page {result.get('page', '?')} | "
                f"Chunk {result.get('chunk_id', '?')} | "
                f"Similarity {score_text}"
            )

            print("-" * 70)
            print(result.get("text", ""))

        continue

    if user_question.lower() == "prompt":

        if last_prompt is None:
            print("No previous prompt is available.")
            continue
```

```
            print("\n" + "=" * 70)
            print("LAST PROMPT SENT TO THE MODEL")
            print("=" * 70)
            print(last_prompt)

            continue

        try:

            answer, retrieved_documents, prompt = rag_query(
                user_question,
                top_k=RAG_TOP_K
            )

            last_answer = answer
            last_sources = retrieved_documents
            last_prompt = prompt

            print("\n" + "=" * 70)
            print("RAG ANSWER")
            print("=" * 70)
            print(answer)

            print("\n" + "=" * 70)
            print("RETRIEVED SOURCES")
            print("=" * 70)

            if not retrieved_documents:
                print("No passages were retrieved.")

            for result in retrieved_documents:

                score = result.get("score", None)
                score_text = (
                    f"{score:.4f}"
                    if isinstance(score, (int, float))
                    else "Unavailable"
                )

                print(
                    f"\n[{result.get('rank', '?')}] "
                    f"PDF Page {result.get('page', '?')} | "
                    f"Chunk {result.get('chunk_id', '?')} | "
                    f"Similarity {score_text}"
                )

                print("-" * 70)
                print(result.get("text", ""))

        except Exception as error:

            print("\nERROR:")
            print(str(error))
            print("The question could not be processed.")
```

```
            print("The question could not be processed.")

    print("\n")
    print("=" * 70)
    print("RAG PROGRAM FINISHED")
    print("=" * 70)
```

Show hidden output

Tested Parameter Configurations

This experiment tested five RAG configurations (A–E) to identify suitable chunk size, overlap, and top-k values for the Barangay VAW Desk Handbook chatbot. Each configuration used consistent documents, test questions, and generation settings. Performance was evaluated based on processing times, answer correctness, retrieval relevance, and source support to determine the most effective balance between answer quality and efficiency.

Run	Chunk Size	Overlap	Top-K	Purpose
A	400	50	3	Evaluate smaller chunks
B	800	100	3	Evaluate medium-sized chunks
C	1,200	150	3	Evaluate larger chunks
D	800	200	3	Evaluate increased overlap
E	800	100	5	Evaluate retrieving more chunks
Selected Configuration	800	80	3	Use the configuration identified as most suitable

---

Compare and Analyze the result CPU and GPU performance

- Comparison between CPU and GPU performance was performed under various settings of RAG system in terms of processing efficiency. In both cases, the experiments were carried out by using the same source documents, test questions, and generation settings. The following parameters were measured: indexing time, average response time, and overall execution time; the quality of answers was assessed by analyzing the generated answers. The obtained data helped to find out how much faster the GPU was and whether its use increased the efficiency of the process.
- Based on the execution times of CPU and GPU provided, it can be seen that

In all test runs the GPU performed faster than the CPU. The overall execution times of CPU ranged from 3 min 3 sec to 4 min 37 sec, while the GPU took the time ranging from 56.29 to 67 sec. The speedup achieved during this process ranged from 3.25× to 4.87×. When speaking about the answer quality, the GPU received Good grade in three tests, while the CPU got the Fair one. However, the difference in the answer quality has to be treated cautiously because it depends only on the processing time.

Experiment Results for Dataset: Barangay VAW Desk Handbook

Device	Chunk Size	Overlap	Top-k	Indexing Time	Average Response Time	Total Execution Time
CPU	400	50	3	1 min 47 sec	30.22 sec per question	4 min 32 sec
GPU	400	50	3	40.27 sec	7.33 sec per question	1 min 6 sec
CPU	800	100	3	1 min 18 sec	30.78 sec per question	4 min 37 sec
GPU	800	100	3	48.062 sec	6.31 sec per question	56.83 sec
CPU	1200	150	3	1 min	20.33 sec per question	3 min 3 sec
GPU	1200	150	3	38.66 sec	6.25 sec per question	56.29 sec
CPU	800	200	3	1 min 1 sec	21.11 sec per question	3 min 16 sec
GPU	800	200	3	36.72 sec	6.3 sec per question	56.70 sec
CPU	800	100	5	1 min 7 sec	27.44 sec per question	4 min 7 sec
GPU	800	100	5	42.74 sec	7.44 sec per question	1 min 7 sec
CPU	800	80	3	58 sec	25.78 sec per question	3 min 58 sec
GPU	800	80	3	40.21 sec	6.4 sec per question	58.3 sec

Performance Metrics and Speedup Calculation

- Average response time is computed as the total time taken to answer the nine test questions divided by the number of questions.

$$\text{Average Response Time} = \frac{\text{Total Time for Answering Questions}}{\text{Number of Questions}}$$

- The speedup was computed using the formula CPU execution time divided by GPU execution time. Then it is categorized as either Speedup greater than 1: GPU is faster, Speedup equal to 1: Both devices have approximately the same execution time, Speedup less than 1: CPU is faster for that measurement.

$$\boxed{\text{Speedup} = \frac{T_{\text{CPU}}}{T_{\text{GPU}}}}$$

[redacted]

Recommended Configuration

- Based on the experimental results, the recommended configuration for the Barangay VAW Desk Handbook Retrieval-Augmented Generation (RAG) chatbot is a chunk size of 800, an overlap of 80, a Top-K value of 3, and GPU inference using google/flan-t5-large. This configuration provided a balance between response time and the ability to generate relevant answers from the handbook.
- During testing, the GPU configuration achieved an average response time of 6.4 seconds per question and a recorded total execution time of 58.3 seconds. In comparison, the CPU configuration recorded an average response time of 25.78 seconds per question and a total execution time of 238 seconds (3 minutes and 58 seconds). Based on the average response times, GPU inference was approximately 4.03 times faster than CPU inference. Using the recorded total execution times, the GPU completed execution approximately 4.08 times faster.
- These results demonstrate the performance advantage of GPU inference for the tested configuration. The chunk size of 800 and overlap of 80 were retained as the selected retrieval settings, while Top-K 3 limits retrieval to three relevant chunks per query. This provides the language model with a focused set of retrieved information to use when generating its responses. However, the selection should be understood in the context of the tested configurations and dataset rather than as a universally optimal setting for all RAG applications.

Evaluation Results of the Barangay VAW Desk Handbook RAG Chatbot

- Evaluation Results of the Barangay VAW Desk Handbook RAG Chatbot The chatbot was evaluated using nine questions derived from the Barangay VAW Desk Handbook. The questions were divided into three difficulty levels: Easy (Q1–Q3), Average (Q4–Q6), and Difficult (Q7–Q9). This evaluation examined whether the chatbot could retrieve relevant handbook information and generate answers that addressed questions about the responsibilities of VAW Desk personnel, the immediate needs of victim-survivors, interview procedures, documentation, and referrals

procedures, documentation, and referrals.

- The generated responses show that the chatbot could provide concise answers to several questions. For example, Q1 identified the role of the VAW Desk in responding to violence cases brought to the barangay, while Q3 identified key information recorded during intake, including personal information, the purpose of the visit, and the action to be taken. Similarly, Q6 recognized the importance of informing victim-survivors about their available rights and remedies. These responses indicate that the chatbot could retrieve and communicate relevant information for several fundamental questions.
- The response to Q7 demonstrated a greater level of detail. It covered the need to provide a safe and private environment, address immediate needs, assess potential risks, arrange medical or protective assistance when necessary, and complete the intake form. This response more closely reflected the interconnected procedures involved in assisting a victim-survivor. In comparison, the responses to Q5 and Q8 were more limited: they identified the need to ask about the victim-survivor's needs or refer the person to appropriate institutions, but did not elaborate on the assessment process or the types of services available. Likewise, the response to Q9 identified the purpose of the VAW Desk Intake Form but did not fully explain how intake, documentation, and referrals work together.

Question ID	Difficulty Level	
Q1	Easy	Responds to woman violence cases brought to the barangay.
Q2	Easy	Food, first aid, and clothing.
Q3	Easy	Personal information; purpose of her visit; action to be taken upon the reques
Q4	Average	Make sure that the victim-survivor is in a stable condition.
Q5	Average	Ask what kind of help she wants.
Q6	Average	Inform the victim/survivor about her rights and remedies available.
Q7	Difficult	Make sure that the victim-survivor is comfortable, in a safe and private room (
Q8	Difficult	The VAW Desk Officer should refer the victim-survivor to the appropriate insti
Q9	Difficult	The VAW Desk Intake Form (Annex A) is completed to obtain basic informatior

Test Questions and Expected Answers

Question ID	Difficulty	Test Question
Q1	Easy	What is the role of the Barangay Violence Against Women (VAW) Desk?
Q2	Easy	What immediate needs should be addressed when a victim-survivor seeks help fron
Q3	Easy	What information should be documented during the interview with a victim-survivor
Q4	Average	What should VAW Desk personnel do before assisting with the interview of a victim-
Q5	Average	What should VAW Desk personnel do when a victim-survivor needs assistance beyo
Q6	Average	What information and assistance should be provided to a victim-survivor regarding

Q7	Difficult	How should VAW Desk personnel address a victim-survivor's immediate needs and c
Q8	Difficult	How should VAW Desk personnel handle a case that requires services from other ins
Q9	Difficult	How do intake procedures, documentation, and referral services work together in as

Initial Observation

- Overall, the results suggest that the chatbot could retrieve relevant information and generate responses addressing the main subject of each of the nine test questions. However, relevance did not always correspond to completeness. Some answers captured the central point of the expected answer but omitted supporting details, procedural steps, or examples of available services. This was particularly noticeable in questions involving referrals and the coordination of multiple assistance procedures.
- The variation in response detail suggests that the chatbot's performance should be evaluated not only by whether its answers address the questions, but also by whether they include the essential information needed to explain the procedures accurately. For a topic such as violence against women and girls, missing procedural details may limit the usefulness of an otherwise relevant response. Therefore, answer evaluation should consider relevance, factual consistency with the handbook, completeness, and the inclusion of important procedural information.
- Regarding computational performance, the selected GPU configuration substantially reduced the recorded response and total execution times compared with CPU execution. This makes GPU inference a practical choice for the tested implementation when faster responses are a priority. Nevertheless, the speed improvement should be interpreted separately from answer quality. The observed differences in answer detail cannot be attributed to GPU acceleration alone, since retrieval settings, the available context, prompt design, and generation parameters can also influence the generated responses.
- Finally, the evaluation was limited to nine test questions and therefore provides an initial indication of the chatbot's performance rather than conclusive evidence of its overall reliability. Further evaluation should include more questions across different difficulty levels, systematic comparison against expected answers, and checks for omitted or unsupported information. Since the chatbot concerns sensitive assistance procedures, its responses should be grounded in the handbook, and users should be directed to qualified personnel or appropriate agencies when further assistance is needed.